T E C H N I C A L A S S I G N M E N T

Employee Leave Management

Multi-Agent System

Domain	Multi-Agent AI Systems
Level	Mid–Senior Engineer
Duration	3–5 Days
Version	1.0

C O N F I D E N T I A L — F O R C A N D I D A T E U S E O N L Y

1. Overview

This assignment challenges you to design and implement a conversational multi-agent system that automates the employee leave application process. The system must orchestrate multiple specialised agents to retrieve employee and policy data, reason about leave entitlements and calendar conflicts, and guide an employee through submitting a confirmed leave request — all through a natural-language interface.

The assignment is intentionally cloud-agnostic. You may deploy your agent runtime on any major cloud provider (Google Cloud Platform, Amazon Web Services, or Microsoft Azure). Your implementation must, however, demonstrate the architectural patterns and behavioural requirements described in this document regardless of the cloud you choose.

2. Business Context

Manual leave management is error-prone, slow, and frequently leads to policy non-compliance. HR teams spend significant time cross-referencing leave balances, public holiday calendars, and approval workflows. This system replaces that friction with an intelligent conversational assistant that:

- Identifies the requesting employee from a persistent data store.
- Fetches the applicable leave policy and available leave balance.
- Retrieves the relevant public holiday calendar to exclude non-working days from the leave count.
- Calculates the exact number of working days consumed by the requested leave window.
- Presents a clear summary to the employee and waits for explicit confirmation before persisting the request.
- Records the confirmed leave application in the database and returns a reference number.

The system must work entirely through conversation — no separate UI is required. All interactions occur through the agent web interface provided by your chosen framework.

3. Functional Requirements

3.1 Core Use Case — Leave Application

When an employee initiates a leave request, the system must:

1. Identify the employee (by ID or name) and retrieve their profile from the SQLite database.
2. Retrieve the employee's current leave balance and applicable leave type (annual, sick, personal, etc.).

3. Retrieve the public holiday calendar for the employee's country/region for the relevant period.
4. Accept a requested leave date range from the employee.
5. Calculate the number of working days in that range, excluding weekends and public holidays.
6. Validate the request against the employee's available balance.
7. Present a structured confirmation summary to the employee including: employee name, leave type, start date, end date, total working days, and remaining balance after approval.
8. Pause and await explicit human confirmation (yes/no) before committing any change.
9. On confirmation, persist the leave record to the database and return a unique reference number.
10. On rejection, discard the request and offer to restart or modify.

3.2 Agent Behaviour Requirements

The system must exhibit the following observable behaviours during the demonstration:

Behaviour	Description	Demonstration Trigger
Sequential Execution	Agents execute in a defined order: employee lookup → policy retrieval → calendar fetch → calculation → confirmation	Single leave application end-to-end
Parallel Execution	Where data retrieval tasks are independent (e.g. policy and holiday calendar), agents execute concurrently to reduce latency	Observe simultaneous tool calls in logs
Human-in-the-Loop	System pauses at the confirmation step and will not proceed until the user provides an explicit approval or rejection	Verify no DB write occurs before confirmation
Database Retrieval	Employee records, leave balances, and leave history are fetched from SQLite via dedicated tool calls	Query employee by ID or name
Computation	Working-day calculation correctly excludes weekends and public holidays across month boundaries	Request spanning a weekend + public holiday
Data Persistence	Approved leave records are written to SQLite with a generated reference ID and updated balance	Verify DB state before and after approval

3.3 Out of Scope

The following are explicitly out of scope for this assignment:

- A custom web or mobile UI — the agent web interface of your chosen framework is sufficient.
- Manager approval workflows or email notifications.
- Authentication, authorisation, or multi-tenant isolation.
- Integration with external HR or payroll systems.
- Real-time calendar sync (a static SQLite holiday table is acceptable).

4. Technical Requirements

4.1 Mandatory Technology Choices

Language	Python 3.12 or higher
Package Manager	UV — all dependencies must be declared in pyproject.toml with a committed uv.lock file. No requirements.txt.
Database	SQLite — local file-based database. Schema must be initialised via a provided SQL script.
Local Testing	The full application must run and be testable using the agent web interface bundled with your chosen framework (e.g. ADK Web for GCP, or equivalent for AWS/Azure). No separate UI is required.

4.2 Agent Framework — Cloud Choice

Select one of the following stacks. You must clearly state your choice in the README and justify it briefly.

Cloud	Agent Framework	Managed Agent Service
GCP	Google ADK 2.0	Vertex AI Agent Engine
AWS	Amazon Bedrock Agents or LangGraph on Bedrock	Amazon Bedrock Agent Runtime
Azure	Azure AI Agent Service SDK or Semantic Kernel	Azure AI Foundry Agent Service

Regardless of cloud, the framework must support:

- Tool / function calling for database access and computation.

- Multi-agent orchestration with at least one orchestrator agent and two or more sub-agents.
- A human-in-the-loop confirmation primitive (interrupt, pause, or equivalent).
- Local development and testing without cloud credentials (or with a free-tier account).

4.3 Agent Architecture

You must implement at minimum the following agent topology:

Agent	Type	Responsibilities
Orchestrator Agent	Root / Coordinator	Receives the user request, routes to sub-agents, manages conversation state, presents confirmation, triggers persistence
Employee Data Agent	Sub-agent / Tool User	Queries SQLite for employee profile, leave balance, and leave history
Calendar Agent	Sub-agent / Tool User	Retrieves public holiday list for the relevant period from SQLite; optionally queries an external API
Leave Calculation Agent	Sub-agent / Pure Logic	Computes working days between two dates, excluding weekends and holidays returned by the Calendar Agent
Leave Persistence Agent	Sub-agent / Tool User	Validates final request, writes approved leave record to SQLite, returns reference ID

You may add additional agents (e.g. a Policy Agent for complex entitlement rules) if your design warrants them. The topology above is a minimum, not a ceiling.

The Orchestrator must coordinate parallel execution between the Employee Data Agent and the Calendar Agent wherever those two retrievals are independent of each other.

5. Data Model

5.1 Required Tables

The SQLite schema must include at least the following tables. Column names are illustrative; you may adjust them as long as the semantic meaning is preserved.

employees

Column	Type	Description	PK/FK
employee_id	TEXT	Unique identifier (e.g. EMP-001)	PK

Column	Type	Description	PK/FK
full_name	TEXT	Employee full name	
department	TEXT	Department or business unit	
country_code	TEXT	ISO 3166-1 alpha-2 (used to look up holiday calendar)	
join_date	DATE	Date of joining (YYYY-MM-DD)	
manager_id	TEXT	Manager employee ID (self-referencing)	FK

leave_balances

Column	Type	Description
balance_id	INTEGER	Auto-increment primary key
employee_id	TEXT	Foreign key to employees
leave_type	TEXT	E.g. ANNUAL, SICK, PERSONAL
year	INTEGER	Calendar year the balance applies to
total_days	REAL	Total entitlement for the year
used_days	REAL	Days consumed to date
available_days	REAL	Computed or maintained: total_days − used_days

public_holidays

Column	Type	Description
holiday_id	INTEGER	Auto-increment primary key
country_code	TEXT	ISO 3166-1 alpha-2
holiday_date	DATE	Date of holiday (YYYY-MM-DD)
holiday_name	TEXT	Human-readable name

leave_requests

Column	Type	Description	
request_id	TEXT	Unique reference number (e.g. LR-2024-0001)	
employee_id	TEXT	Foreign key to employees	

Column	Type	Description	
leave_type	TEXT	Leave type requested	
start_date	DATE	Inclusive start date	
end_date	DATE	Inclusive end date	
working_days	REAL	Calculated working days consumed	
status	TEXT	PENDING | APPROVED | REJECTED | CANCELLED	
submitted_at	DATETIME	Timestamp of submission	
notes	TEXT	Optional employee notes	

5.2 Seed Data Requirements

Your database initialisation script must populate:

- At least 5 employee records across at least 2 different country codes.
- Leave balances for each employee for the current calendar year, covering at least ANNUAL and SICK leave types.
- Public holidays for at least 2 countries covering the current year (minimum 8 holidays per country).
- At least 2 existing leave requests (one approved, one pending) to demonstrate history retrieval.

6. Deliverables

Submit all deliverables as a single Git repository (GitHub, GitLab, or Bitbucket). The repository root must contain:

Deliverable	Acceptance Criteria
Source Code	All agent definitions, tool implementations, orchestration logic. Code must be importable and runnable via UV without additional manual setup steps beyond those in the README.
init_db.sql (or init_db.py)	Creates all tables and inserts seed data. Running this script from scratch must produce a fully functional database. Idempotent (safe to re-run).
README.md	Step-by-step local setup, environment variable list, how to run ADK Web (or equivalent), how to run the cloud deployment, and any

Deliverable	Acceptance Criteria
	known limitations.
SAMPLE_PROMPTS.md	At least 6 sample prompts with expected agent behaviour. Must include prompts that exercise all six demonstration scenarios listed in Section 3.2.
Architecture Diagram	A diagram (image or Mermaid/PlantUML in a .md file) showing the agent topology, tool call flow, and database interactions.
Cloud Deployment Config	Infrastructure-as-code or CLI scripts (e.g. cloudbuild.yaml, CDK stack, Bicep template) sufficient to deploy the agent to the managed service of your chosen cloud.

7. Demonstration Requirements

Your submission must be accompanied by a live or recorded demonstration that explicitly covers all six scenarios below. If recorded, the video must be unedited and show the agent web interface in real time.

Scenario 1 — Sequential Execution

Prompt the agent to apply for leave for a named employee. Walk through the full flow: employee lookup → policy check → calendar fetch → calculation → confirmation summary. The evaluator must be able to observe each agent step in the framework's trace or log output.

Scenario 2 — Parallel Execution

Demonstrate (via framework logs or a trace panel) that the Employee Data Agent and the Calendar Agent are invoked concurrently rather than sequentially when processing a leave request.

Scenario 3 — Human-in-the-Loop

Show that after the system presents the confirmation summary, it waits indefinitely. First, reject the request and verify no database write occurred. Then submit a new request and approve it. Verify the database record.

Scenario 4 — Database Retrieval

Ask the agent for an employee's current leave balance and history without initiating a new request. The agent must query and return structured data from SQLite.

Scenario 5 — Computation

Request leave spanning a date range that crosses at least one weekend and at least one public holiday. The system must return the correct number of working days (not calendar days). Verify by manual count.

Scenario 6 — Data Persistence

Before submitting a request, query the database to show current leave balance and request count. Submit and approve a request. Query again to confirm the leave_requests table has a new row, used_days has increased, and available_days has decreased accordingly.

8. Evaluation Criteria

Criterion	What Evaluators Look For	Weight
Agent Architecture	Clear separation of concerns, appropriate agent granularity, well-defined tool interfaces	25%
Orchestration Patterns	Correct sequential flow, demonstrable parallel execution, proper human-in-the-loop implementation	25%
Code Quality	Readable, modular Python; typed interfaces; meaningful error handling; UV/pyproject.toml compliance	20%
Data Layer	Correct schema, idempotent init script, accurate working-day calculation, proper persistence	15%
Documentation & Runability	README enables a fresh setup in under 15 minutes; sample prompts cover all scenarios	10%
Cloud Deployment	Agent successfully deployed to and invokable from the managed agent service of chosen cloud	5%

Submissions that do not run locally via the agent web interface will not be evaluated for orchestration pattern criteria, regardless of code quality.

9. Sample Prompts

The following prompts are provided as a starting point. Your SAMPLE_PROMPTS.md must include these and may add more.

Prompt 1 — Full Leave Application (Sequential + Parallel + Human-in-the-Loop)

```
I’d like to apply for annual leave from 14 July 2025 to 25 July 2025 for
employee EMP-003.
```

Expected behaviour: agent retrieves employee details and holiday calendar concurrently, calculates working days (excluding weekends and any public holidays in that window), presents a confirmation summary, and waits for approval.

Prompt 2 — Balance Enquiry (Database Retrieval)

```
What is the current leave balance for Sarah Nguyen?
```

Expected behaviour: agent queries the database for all leave types for the named employee and returns a structured summary without entering the application flow.

Prompt 3 — Computation Verification (Working-Day Calculation)

```
How many working days are there between 23 December 2025 and 2 January 2026
for an employee based in GB?
```

Expected behaviour: agent retrieves the UK holiday calendar, excludes 25–26 December and 1 January plus all weekends, and returns the correct working-day count.

Prompt 4 — Rejection Flow (Human-in-the-Loop)

```
[After the system presents a confirmation summary] No, cancel that request.
I made an error with the dates.
```

Expected behaviour: agent discards the pending request (no database write), acknowledges the cancellation, and offers to start again.

Prompt 5 — Leave History (Database Retrieval)

```
Show me all approved leave requests for EMP-001 this year.
```

Expected behaviour: agent queries leave_requests table filtered by employee and status, returns a structured list including dates, type, and working days consumed.

Prompt 6 — Insufficient Balance (Validation)

```
Apply for 30 days of annual leave for EMP-002 starting 1 September 2025.
```

Expected behaviour: agent calculates the working days, checks the balance, determines insufficient entitlement, and returns a clear error message without entering the confirmation flow.

10. Submission Instructions

10.1 Repository

- Host the repository on GitHub, GitLab, or Bitbucket.
- The default branch must be named main.
- Do not commit .env files, API keys, or cloud credentials.
- Include a .gitignore that excludes the SQLite database file, __pycache__, .venv, and any IDE configuration.

10.2 Required Files at Repository Root

File / Directory	Purpose
pyproject.toml	UV project manifest with all dependencies
uv.lock	Committed lockfile for reproducible installs
init_db.sql or init_db.py	Schema creation and seed data
README.md	Setup and run instructions
SAMPLE_PROMPTS.md	Six or more sample prompts with expected behaviour
src/ or agents/	Python source code
deploy/ or infra/	Cloud deployment scripts / IaC
docs/architecture.*	Architecture diagram

10.3 Deadline & Queries

Submit the repository URL to the hiring team by the date communicated in your assignment email. For technical clarifications, raise a GitHub Issue on the repository and tag it question — the evaluator will respond within one business day.

11. Appendix — Suggested Project Structure

The structure below is a recommendation, not a requirement. Adapt it to the conventions of your chosen framework.

```
leave-management-agent/
├── pyproject.toml
├── uv.lock
├── init_db.sql
├── README.md
├── SAMPLE_PROMPTS.md
├── src/
│   ├── agents/
│   │   ├── orchestrator.py
│   │   ├── employee_data_agent.py
│   │   ├── calendar_agent.py
│   │   ├── calculation_agent.py
│   │   └── persistence_agent.py
│   ├── tools/
│   │   ├── db_tools.py
│   │   └── date_tools.py
│   └── database/
│       └── connection.py
├── deploy/
│   └── [cloud-specific IaC / deployment scripts]
└── docs/
    └── architecture.md
```

Note: if your chosen framework uses a different project layout convention (e.g. ADK's agent.py + .adk/ directory model), follow that convention and note any deviations from the above structure in your README.